6G Slicing — Two-Tier Control Plane: Full Analysis

Prototype report. Dataset: Farreras et al. 2024 (Zenodo 10610616). Analytic capacity model + per-slice LLM/MCP compiler.

1. Dataset (EDA)

100 samples. Aggregate load 47–810 Mbps (mean 376).

Head (first 10 samples):

eMBB Mbps	URLLC Mbps	mMTC Mbps	delay	loss
36	7	4	0.02189	0.567
36	32	5	0.03590	0.572
36	24	14	0.02249	0.634
54	21	2	0.01953	0.361
36	46	2	0.00242	0.005
72	21	2	0.00912	0.057
36	60	3	0.00641	0.047
36	49	14	0.00378	0.081
72	24	26	0.01010	0.338
72	49	6	0.00335	0.058

Per-slice load:

slice	min load	max load	mean load	% of total
eMBB	36	378	174	46%
URLLC	7	382	170	45%
mMTC	2	132	32	9%

KPI: delay min=0.00021 max=0.03590 mean=0.00488 | loss min=0.005 max=0.843 mean=0.253

2. Tier-1 components + KPI budgets

• **Capacity model** — analytic, deterministic (not ML); the scaling math every other part trusts:

```
effective_capacity = replicas * cap_per_replica
rho = load / capacity
delay = base_delay / (1 - rho)  if rho < 1 else max_delay
loss  = (load - capacity) / load if overloaded else 0
violation = delay > delay_budget OR loss > loss_budget
required_replicas = ceil(load / (cap * target_util))
```

• calibrated from the slice's P95 load (cap) and the dataset's min delay (base_delay); feeds the env, oracle, gate, baselines and MCP tools.

• **Forecaster** — learned. Predicts next aggregate load: linear regression over the last 5 points (or persistence). Output: forecast_mbps.

• **KPI predictor** — learned. Numpy MLP regressor (16 hidden, ReLU, MSE) trained on aggregate load → delay. p_violation = sigmoid((pred_delay − budget)/budget).

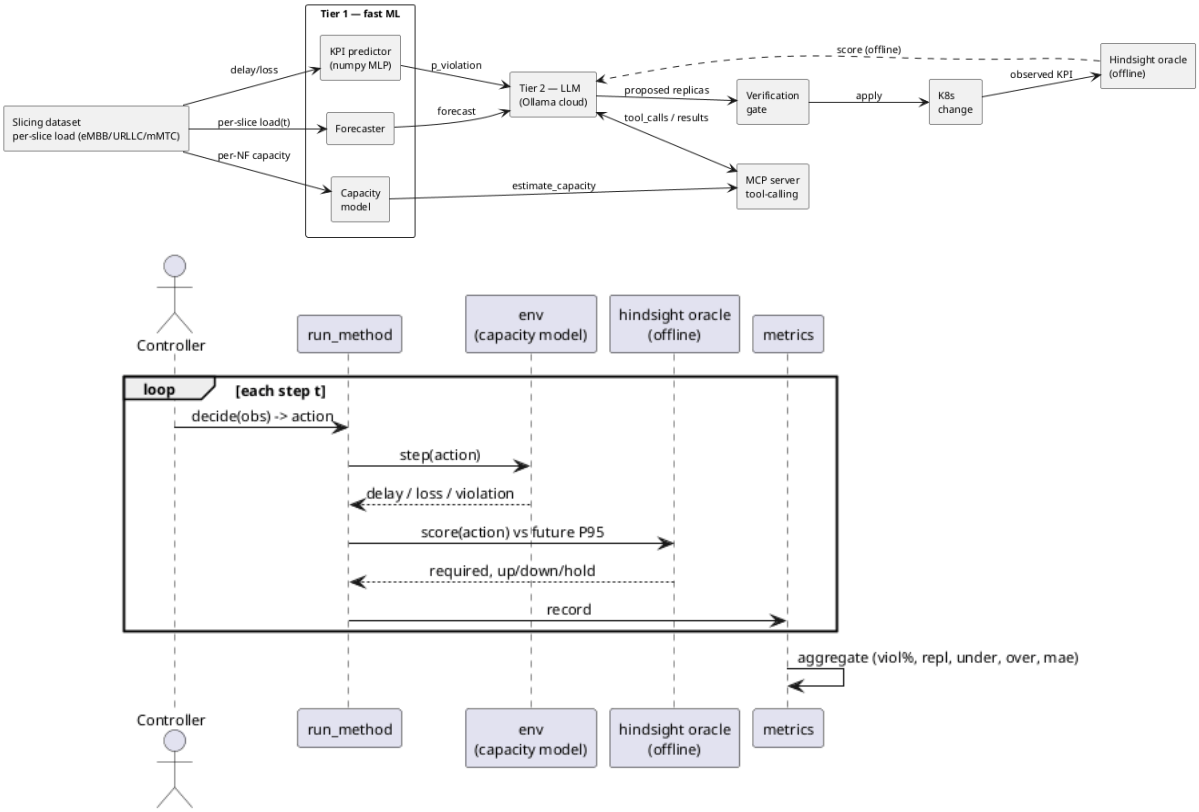
slice	cap/replica Mbps	delay_budget (x base)	loss_budget
eMBB	98	30x	0.02
URLLC	101	4x	0.005
mMTC	29	15x	0.005

3. Pipeline + harness

• dataset sample -> per-slice load -> Tier1 (forecaster + MLP) -> payload -> LLM (or capacity-rule) -> gate -> apply

• hindsight oracle (offline): future-load P95 -> required replicas, scores under/over-scaling

• every controller is decide(obs) -> action; run_method feeds it to env + oracle; metrics() aggregates



4. Methods

- **threshold** — reactive: scale only after a violation.
- **hpa** — ideal controller, always required_replicas(load).
- **predict-then-threshold** — MLP delay prediction then threshold (degenerates to HPA).
- **capacity-rule** — deterministic rule, near-optimal reference (shares the capacity model).
- **llm-real** — Ollama cloud LLM, JSON or MCP tool-calling.

5. LLMs, prompt, tools

Models (Ollama cloud, localhost, OpenAI-compatible, temp 0): glm-5.3-flash:cloud, glm-5.3:cloud, deepseek-v4-pro:cloud

Note: mistral-large-4 is excluded — ~3x slower and ~2x more tokens than the others while returning the same decisions.

System prompt (tool mode):

```
You are the Tier-2 compiler of a 6G network-slicing control plane. You receive per-slice telemetry
(eMBB, URLLC, mMTC): load (Mbps), current replicas, current delay/loss, KPI targets, and ML signals.
Use the provided tools to compute the correct replica count. Then act: call scale_replicas once per
slice (eMBB, URLLC, mMTC) with the replica count you decide. Do not output text or JSON - your
scale_replicas calls are the answer.
```

tool	does
estimate_capacity(slice, load)	load -> required replicas (capacity model)
scale_replicas(slice, n)	validate + clamp n to [1,5]
set_resources(slice, cpu, mem)	resource-limit manifest patch
add_nf_instance(slice, region)	register a new NF instance
apply_k8s_manifest(id, yaml)	validate a Deployment manifest

inspect_infrastructure(state)	utilization, capacity, headroom, required, budgets per slice

6. Function-call traces (sample)

glm-5.3-flash:cloud (state i=0) — final actions {'eMBB': 2, 'URLLC': 1, 'mMTC': 1}

tool call	arguments	result
inspect_infrastructure	state (3 slices)	eMBB: util=0.368 req=1 URLLC: util=0.069 req=1 mMTC: util=0.154 req=1
scale_replicas	eMBB -> 2	ok
scale_replicas	URLLC -> 1	ok
scale_replicas	mMTC -> 1	ok

deepseek-v4-pro:cloud (state i=0) — final actions {'eMBB': 1, 'URLLC': 1, 'mMTC': 1}

tool call	arguments	result
inspect_infrastructure	state (3 slices)	eMBB: util=0.368 req=1 URLLC: util=0.069 req=1 mMTC: util=0.154 req=1
scale_replicas	eMBB -> 1	ok
scale_replicas	URLLC -> 1	ok
scale_replicas	mMTC -> 1	ok

glm-5.3:cloud (state i=0) — final actions {'eMBB': 5, 'URLLC': 5, 'mMTC': 5}

tool call	arguments	result
inspect_infrastructure	state (3 slices)	eMBB: util=0.368 req=1 URLLC: util=0.069 req=1 mMTC: util=0.154 req=1
scale_replicas	eMBB -> 5	ok
scale_replicas	URLLC -> 5	ok
scale_replicas	mMTC -> 5	ok
scale_replicas	eMBB -> 5	ok

7. Benchmark (per-slice closed loop)

Scenario: **ramp**

method	viol%	repl	under	over	mae
threshold	24.67	2.50	1.11	0.11	0.49
hpa	15.67	2.63	0.95	0.08	0.00
predict-then-threshold	15.67	2.63	0.95	0.08	0.00
capacity-rule	1.00	3.73	0.16	0.38	1.10

Scenario: **steady**

method	viol%	repl	under	over	mae
threshold	27.50	2.58	1.28	0.22	0.59
hpa	16.67	2.63	1.13	0.12	0.00
predict-then-threshold	16.67	2.63	1.13	0.12	0.00
capacity-rule	0.83	4.28	0.12	0.76	1.65

Scenario: **spike**

method	viol%	repl	under	over	mae
threshold	20.00	2.15	1.11	0.19	0.44
hpa	14.58	2.24	0.95	0.12	0.00
predict-then-threshold	15.00	2.23	0.96	0.12	0.01
capacity-rule	1.67	4.06	0.22	1.21	1.82

Scenario: **ramp-llm**

method	viol%	repl	under	over	mae
threshold	28.43	2.49	1.47	0.10	0.50

hpa	20.59	2.52	1.41	0.07	0.00
predict-then-threshold	20.59	2.52	1.41	0.07	0.00
capacity-rule	2.94	3.55	0.49	0.18	1.03
llm-real-flash	8.82	3.63	0.81	0.58	1.26

8. RQ5 — estimate_capacity tool

mode	viol%	repl	mae	under	over
no_tool	3.33	3.53	1.22	0.50	0.48
tool	10.00	2.57	0.22	1.12	0.13

Finding: tool cuts MAE 1.18->0.20 and cost 3.47->2.55, but raises violation 6.7%->11.7% (exact, not safer).

9. Inspect tool A/B — per scenario (mistral excluded)

Scenario: **ramp** (over/under/mae vs required_replicas)

model	arm	over	under	mae
glm-5.3-flash:cloud	no-tool	0.23	0.30	0.53
glm-5.3-flash:cloud	inspect	0.07	0.00	0.07
deepseek-v4-pro:cloud	no-tool	0.83	0.07	0.90
deepseek-v4-pro:cloud	inspect	0.00	0.10	0.10
glm-5.3:cloud	no-tool	0.87	0.03	0.90
glm-5.3:cloud	inspect	0.03	0.00	0.03

Scenario: **steady** (over/under/mae vs required_replicas)

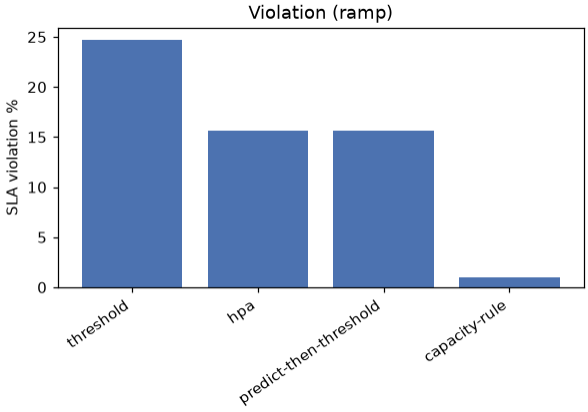
model	arm	over	under	mae
glm-5.3-flash:cloud	no-tool	0.60	0.27	0.87
glm-5.3-flash:cloud	inspect	0.00	0.00	0.00
deepseek-v4-pro:cloud	no-tool	1.13	0.03	1.17
deepseek-v4-pro:cloud	inspect	0.00	0.00	0.00
glm-5.3:cloud	no-tool	0.93	0.10	1.03
glm-5.3:cloud	inspect	0.03	0.00	0.03

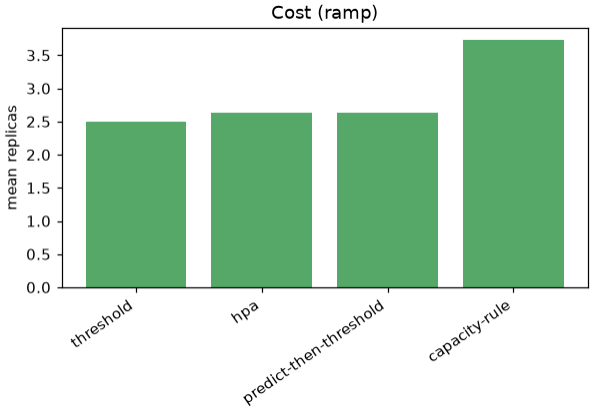
Scenario: **spike** (over/under/mae vs required_replicas)

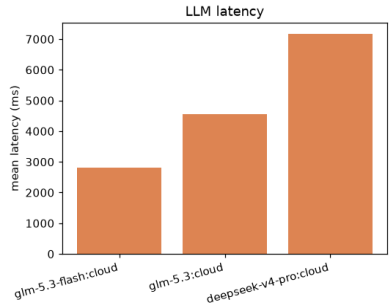
model	arm	over	under	mae
glm-5.3-flash:cloud	no-tool	0.47	0.40	0.87
glm-5.3-flash:cloud	inspect	0.07	0.07	0.13
deepseek-v4-pro:cloud	no-tool	0.80	0.10	0.90
deepseek-v4-pro:cloud	inspect	0.03	0.00	0.03
glm-5.3:cloud	no-tool	0.97	0.20	1.17
glm-5.3:cloud	inspect	0.11	0.00	0.11

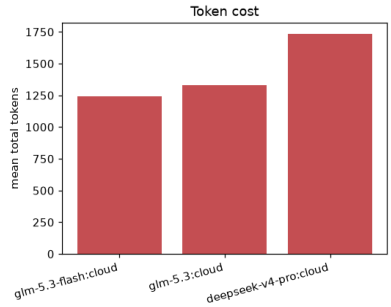
Finding: the tool makes capable models near-exact (deepseek and glm drop MAE sharply) but flash regresses. Direction holds across ramp, steady and spike.

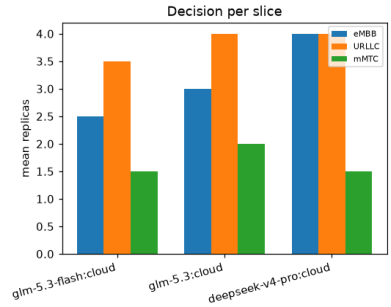
10. Comparison graphs


Violation (ramp)
SLA violation %
25
20
15
10
5
0
threshold
hpa
predict-then-threshold
capacity-rule


Cost (ramp)
mean replicas
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
threshold
hpa
predict-then-threshold
capacity-rule


LLM latency
mean latency (ms)
7000
6000
5000
4000
3000
2000
1000
0
glm-5.3-flash:cloud
glm-5.3:cloud
deepseek-v4-pro:cloud


Token cost
mean total tokens
1750
1500
1250
1000
750
500
250
0
glm-5.3-flash:cloud
glm-5.3:cloud
deepseek-v4-pro:cloud


Decision per slice
mean replicas
eMBB
URLLC
mMTC
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
glm-5.3-flash:cloud
glm-5.3:cloud
deepseek-v4-pro:cloud